

Veritas

AI-Powered Misinformation Detection - Source Code

BSc Data Science Third Year Project | August 2026

Contents

01. `src/convex/analyzeNews.ts`

NLP engine v3 - category breakdowns, keyword detection

02. `src/convex/schema.ts`

Database schema with breakdown fields

03. `src/convex/analyses.ts`

Backend CRUD operations

04. `src/components/CredibilityGauge.tsx`

Animated SVG confidence gauge

05. `src/components/StatsView.tsx`

Statistics with Recharts

06. `src/components/MethodologyView.tsx`

Academic methodology docs

07. `src/components/ThemeProvider.tsx`

Dark mode provider

08. `src/pages/Dashboard.tsx`

Main app - 10 improvements integrated

```

1  "use node";
2
3  import { action } from "../generated/server";
4  import { v } from "convex/values";
5
6  // -----
7  // Veritas NLP-Based Fake News Detection Engine v3
8  // Now returns category-level breakdowns for visualization.
9  // -----
10
11 const SENSATIONALIST: Array<RegExp, number> = [
12   [/[A-Z]{3,}!{2,}/g, 8], [/shocking|unbelievable|mind[- ]?blowing/gi, 6],
13   [/miracle|wonder|amazing|incredible/gi, 5], [/urgent|breaking|just in/gi, 4],
14   [/exposed|revealed/gi, 6], [/secret|hidden|suppressed|banned|censored/gi, 6],
15   [/cover[- ]?up|conspiracy/gi, 7], [/they don't want you to know/gi, 8],
16   [/share (this|before|now)|before they delete/gi, 7],
17   [/wake up|open your eyes|do your research/gi, 6],
18   [/the truth (about|they|is)|what they're hiding/gi, 7],
19   [/mainstream media (won't|doesn't|is paid)/gi, 8],
20   [/you won't (believe|want to miss)/gi, 7],
21 ];
22
23 const CLICKBAIT: Array<RegExp, number> = [
24   [/you won't believe/i, 6], [/doctors? (don't|hate|are shocked)/i, 7],
25   [/one|a (trick|simple|weird|secret)/i, 5],
26   [/number \d+ will (shock|amaze|surprise)/i, 7],
27   [/d+%s*of\s*(people|doctors)\s*(don't|won't)/i, 7],
28   [/before it's|while you still can)/i, 5],
29   [/click here|act now|limited time)/i, 6],
30   [/celebrities|hate|love) this (trick|secret)/i, 7],
31 ];
32
33 const ANONYMOUS_SOURCING: Array<RegExp, number> = [
34   [/experts? (say|claim|believe|warn)/gi, 4], [/studies (show|reveal|suggest|prove)/gi, 3],
35   [/scientists? (say|warn|discover|reveal)/gi, 3], [/insiders? (reveal|say|claim)/gi, 5],
36   [/sources? (say|claim|reveal)/gi, 4], [/anonymous (source|insider|official)/gi, 7],
37   [/people|everyone) (are saying|say|claim)/gi, 5],
38 ];
39
40 const FEAR_MONGERING: Array<RegExp, number> = [
41   [/the elite|the powerful) (are|want|plan|will) to/gi, 5],
42   [/big (pharma|tech|media|tobacco|oil)/gi, 6],
43   [/deep state|new world order|illuminati/gi, 7],
44   [/mind control|brainwash|programming/gi, 6],
45   [/depopulation|eugenics|population control/gi, 7],
46 ];
47
48 const CONSPIRACY: Array<RegExp, number> = [
49   [/conspiracy|conspir(acy|ies|ing)/gi, 6], [/cover[- ]?up|covering up/gi, 6],
50   [/wake up|open your eyes/gi, 6], [/do your (own)?research/gi, 5],
51   [/sheeple/gi, 7], [/fake news|lamestream/gi, 5],
52   [/deep state|shadow government/gi, 7], [/globalist|global elites|cabal/gi, 6],
53   [/censored|silenced|suppressed/gi, 5],
54 ];
55
56 const CREDIBLE_INDICATORS: Array<RegExp, number> = [
57   [/according to (the |a |research |official )/gi, 5],
58   [/published (in|on|by)/gi, 6], [/study|research|report) (published|found|conducted)/gi, 6],
59   [/data|findings) (from|show|indicate)/gi, 5],
60   [/officials?|spokesperson) (said|stated|confirmed)/gi, 5],
61   [/in a (press|public|official) statement/gi, 6],
62   [/CEO|CTO|CFO|president|director|minister)/gi, 4],
63   [/peer[- ]?reviewed|peer reviewed)/gi, 7],
64   [/found that|showed that|revealed that|confirmed that)/gi, 4],
65 ];
66
67 const CREDIBLE_SOURCES = [
68   /\breuters\b/i, /\bbc\b/i, /\bnew york times\b/i, /\bthe guardian\b/i,
69   /\bnature\b/i, /\bscience\b/i, /\bthe lancet\b/i, /\bjournal of\b/i,

```

```

70     /\buniversity of\b/i, /\bnasa\b/i, /\bcd\b/i, /\bfda\b/i,
71     /\bharvard\b/i, /\bmit\b/i, /\bstanford\b/i, /\boxford\b/i,
72 ];
73
74 // --- KEYWORD HIGHLIGHTING -----
75
76 function findTriggeredKeywords(text: string): string[] {
77     const keywords: string[] = [];
78     const allPatterns = [
79         ...SENSATIONALIST, ...CLICKBAIT, ...ANONYMOUS_SOURCING,
80         ...FEAR_MONGERING, ...CONSPIRACY,
81     ];
82     for (const [pattern] of allPatterns) {
83         const matches = text.match(pattern);
84         if (matches) {
85             for (const m of matches.slice(0, 2)) {
86                 const clean = m.trim();
87                 if (clean.length > 2 && !keywords.includes(clean)) keywords.push(clean);
88             }
89         }
90     }
91     return keywords.slice(0, 15);
92 }
93
94 // --- WEIGHTED MATCHING -----
95
96 function weightedMatches(text: string, patterns: Array<[RegExp, number]>) {
97     let total = 0, score = 0;
98     const matches: string[] = [];
99     for (const [pattern, weight] of patterns) {
100         const found = text.match(pattern);
101         if (found) {
102             total += found.length;
103             score += found.length * weight;
104             matches.push(...found.slice(0, 2).map(m => m.trim()));
105         }
106     }
107     return { total, score, matches: [...new Set(matches)].slice(0, 3) };
108 }
109
110 // --- ANALYSIS -----
111
112 function analyzeText(text: string) {
113     const wordCount = text.split(/\s+/).length;
114     let redFlagScore = 0, greenFlagScore = 0;
115     const redFlags: string[] = [], greenFlags: string[] = [];
116     const categoryBreakdown: Array<{ category: string; type: "red" | "green"; score: number; maxS
117
118     // -- RED FLAGS --
119     const redCategories = [
120         { name: "Sensationalism", patterns: SENSATIONALIST, threshold1: 30, threshold2: 15, msg1: (
121         { name: "Clickbait", patterns: CLICKBAIT, threshold1: 15, threshold2: 8, msg1: (n: number)
122         { name: "Anonymous Sourcing", patterns: ANONYMOUS_SOURCING, threshold1: 20, threshold2: 10,
123         { name: "Fear-Mongering", patterns: FEAR_MONGERING, threshold1: 20, threshold2: 10, msg1: "
124         { name: "Conspiracy", patterns: CONSPIRACY, threshold1: 18, threshold2: 9, msg1: (n: number
125     ];
126
127     for (const cat of redCategories) {
128         const result = weightedMatches(text, cat.patterns);
129         let score = 0, findings: string[] = [];
130         if (result.score >= cat.threshold1) { score = 30; findings.push(typeof cat.msg1 === "functi
131         else if (result.score >= cat.threshold2) { score = 15; findings.push(typeof cat.msg2 === "f
132         redFlagScore += score;
133         if (findings.length) redFlags.push(...findings);
134         categoryBreakdown.push({ category: cat.name, type: "red", score, maxScore: 30, findings });
135     }
136
137     // Additional red flags
138     const capsCount = (text.match(/[A-Z]{3,!{1,}/g))?.length ?? 0;
139     if (capsCount >= 5) { redFlagScore += 18; redFlags.push(`Excessive caps/exclamation (${capsCo
140
141     const emojiCount = (text.match(/[\u{1F600}-\u{1F64F}\u{1F300}-\u{1F5FF}\u{1F680}-\u{1F6FF}]/g
142     if (emojiCount >= 3) { redFlagScore += 12; redFlags.push(`Excessive emoji (${emojiCount})`);
143

```

```

144     if (text.match(/share.*(before|now)|before.*(delete|remove)/i)) { redFlagScore += 12; redFlag
145
146     // -- GREEN FLAGS --
147     const cred = weightedMatches(text, CREDIBLE_INDICATORS);
148     let credScore = 0;
149     if (cred.score >= 25) { credScore = 25; greenFlags.push(`Strong sourcing (${cred.total})`); }
150     else if (cred.score >= 12) { credScore = 16; greenFlags.push(`Good sourcing (${cred.total})`)
151     greenFlagScore += credScore;
152     categoryBreakdown.push({ category: "Source Quality", type: "green", score: credScore, maxScor
153
154     let statsScore = 0;
155     if (/(\d+%|\${\d,}+|\d+ million|\d+ billion/i.test(text)) { statsScore = 16; greenFlags.push("
156     greenFlagScore += statsScore;
157     categoryBreakdown.push({ category: "Data & Statistics", type: "green", score: statsScore, max
158
159     const credCount = CREDIBLE_SOURCES.filter(p => p.test(text)).length;
160     let instScore = 0;
161     if (credCount >= 2) { instScore = 22; greenFlags.push(`Credible institutions (${credCount})`)
162     greenFlagScore += instScore;
163     categoryBreakdown.push({ category: "Institutional References", type: "green", score: instScor
164
165     let dateScore = 0;
166     if (/on (monday|tuesday|wednesday|thursday|friday)/i.test(text)) { dateScore = 14; greenFlags
167     greenFlagScore += dateScore;
168     categoryBreakdown.push({ category: "Temporal Specificity", type: "green", score: dateScore, m
169
170     let balanceScore = 0;
171     if (/however|on the other hand|critics (say|argue)/i.test(text)) { balanceScore = 18; greenFl
172     greenFlagScore += balanceScore;
173     categoryBreakdown.push({ category: "Balanced Reporting", type: "green", score: balanceScore,
174
175     let structScore = 0;
176     const struct = ["who", "what", "where", "when", "why"].filter(w => new RegExp(`\\b${w}\\b`, "
177     if (struct >= 4) { structScore = 16; greenFlags.push("Journalistic structure"); }
178     greenFlagScore += structScore;
179     categoryBreakdown.push({ category: "Journalistic Structure", type: "green", score: structScor
180
181     let lenScore = 0;
182     if (wordCount >= 80 && wordCount <= 800) { lenScore = 6; greenFlags.push("Appropriate length"
183     greenFlagScore += lenScore;
184
185     // -- VERDICT --
186     const total = Math.max(redFlagScore + greenFlagScore, 1);
187     const redRatio = redFlagScore / total, greenRatio = greenFlagScore / total;
188     let verdict: "likely_real" | "likely_fake" | "uncertain";
189     let confidence: number;
190
191     if (redRatio >= 0.65) { verdict = "likely_fake"; confidence = Math.min(95, Math.round(55 + re
192     else if (greenRatio >= 0.65) { verdict = "likely_real"; confidence = Math.min(95, Math.round(
193     else if (redRatio > greenRatio + 0.1) { verdict = "likely_fake"; confidence = Math.min(78, Ma
194     else if (greenRatio > redRatio + 0.1) { verdict = "likely_real"; confidence = Math.min(78, Ma
195     else { verdict = "uncertain"; confidence = Math.round(35 + Math.abs(redRatio - greenRatio) *
196     confidence = Math.max(35, Math.min(95, confidence));
197
198     const triggeredKeywords = findTriggeredKeywords(text);
199
200     const summary = verdict === "likely_fake"
201       ? `This content shows ${redFlags.length} red flags suggesting it may be unreliable. ${redFl
202       : verdict === "likely_real"
203       ? `This content shows ${greenFlags.length} indicators consistent with credible journalism.
204       : `Mixed profile: ${redFlags.length} concerns, ${greenFlags.length} positives. Verify throu
205
206     const parts: string[] = [];
207     if (redFlags.length) parts.push(`Concerns: ${redFlags.slice(0, 3).join("; ")}`);
208     if (greenFlags.length) parts.push(`Positives: ${greenFlags.slice(0, 3).join("; ")}`);
209     parts.push(`Confidence: ${confidence}% based on linguistic pattern analysis, source verificat
210
211     return {
212       verdict, confidence, summary, redFlags, greenFlags,
213       reasoning: parts.join(" "),
214       triggeredKeywords,
215       categoryBreakdown: categoryBreakdown.filter(c => c.maxScore > 0),
216       wordCount,
217     };

```

```
218   }
219
220   export const analyzeNews = action({
221     args: { text: v.string(), inputType: v.union(v.literal("text"), v.literal("url")) },
222     handler: async (_ctx, args) => {
223       if (args.text.trim().length < 10) {
224         return {
225           verdict: "uncertain" as const, confidence: 30,
226           summary: "Text too short for meaningful analysis.",
227           redFlags: ["Too short"], greenFlags: [],
228           reasoning: "Minimum content required.", triggeredKeywords: [],
229           categoryBreakdown: [], wordCount: args.text.trim().split(/\s+/).length,
230         };
231       }
232       return analyzeText(args.text.trim());
233     },
234   });
235
```

```

1  import { authTables } from "@convex-dev/auth/server";
2  import { defineSchema, defineTable } from "convex/server";
3  import { Infer, v } from "convex/values";
4
5  // default user roles. can add / remove based on the project as needed
6  export const ROLES = {
7    ADMIN: "admin",
8    USER: "user",
9    MEMBER: "member",
10 } as const;
11
12 export const roleValidator = v.union(
13   v.literal(ROLES.ADMIN),
14   v.literal(ROLES.USER),
15   v.literal(ROLES.MEMBER),
16 );
17 export type Role = Infer<typeof roleValidator>;
18
19 const schema = defineSchema(
20   {
21     // default auth tables using convex auth.
22     ...authTables, // do not remove or modify
23
24     // the users table is the default users table that is brought in by the authTables
25     users: defineTable({
26       name: v.optional(v.string()),
27       image: v.optional(v.string()),
28       email: v.optional(v.string()),
29       emailVerificationTime: v.optional(v.number()),
30       isAnonymous: v.optional(v.boolean()),
31       role: v.optional(roleValidator),
32     }).index("email", ["email"]),
33
34     // Analysis history table
35     analyses: defineTable({
36       userId: v.string(),
37       inputText: v.string(),
38       inputType: v.union(v.literal("text"), v.literal("url")),
39       verdict: v.union(
40         v.literal("likely_real"),
41         v.literal("likely_fake"),
42         v.literal("uncertain"),
43       ),
44       confidence: v.number(),
45       summary: v.string(),
46       redFlags: v.array(v.string()),
47       greenFlags: v.array(v.string()),
48       reasoning: v.string(),
49       triggeredKeywords: v.optional(v.array(v.string())),
50       categoryBreakdown: v.optional(v.array(v.object({
51         category: v.string(),
52         type: v.union(v.literal("red"), v.literal("green")),
53         score: v.number(),
54         maxScore: v.number(),
55         findings: v.array(v.string()),
56       }))),
57       wordCount: v.optional(v.number()),
58       createdAt: v.number(),
59     }).index("by_user", ["userId", "createdAt"]),
60   },
61   {
62     schemaValidation: false,
63   },
64 );
65
66 export default schema;
67

```

```
1  import { query, mutation } from "../_generated/server";
2  import { v } from "convex/values";
3
4  export const listByUser = query({
5    args: {},
6    handler: async (ctx) => {
7      const identity = await ctx.auth.getUserIdentity();
8      if (!identity) return [];
9      const analyses = await ctx.db
10        .query("analyses")
11        .withIndex("by_user", (q) => q.eq("userId", identity.subject))
12        .order("desc")
13        .take(50);
14      return analyses;
15    },
16  });
17
18  export const get = query({
19    args: { id: v.id("analyses") },
20    handler: async (ctx, args) => {
21      const identity = await ctx.auth.getUserIdentity();
22      if (!identity) return null;
23      const analysis = await ctx.db.get(args.id);
24      if (!analysis || analysis.userId !== identity.subject) return null;
25      return analysis;
26    },
27  });
28
29  export const create = mutation({
30    args: {
31      inputText: v.string(),
32      inputType: v.union(v.literal("text"), v.literal("url")),
33      verdict: v.union(
34        v.literal("likely_real"),
35        v.literal("likely_fake"),
36        v.literal("uncertain"),
37      ),
38      confidence: v.number(),
39      summary: v.string(),
40      redFlags: v.array(v.string()),
41      greenFlags: v.array(v.string()),
42      reasoning: v.string(),
43    },
44    handler: async (ctx, args) => {
45      const identity = await ctx.auth.getUserIdentity();
46      if (!identity) throw new Error("Not authenticated");
47      return ctx.db.insert("analyses", {
48        userId: identity.subject,
49        ...args,
50        createdAt: Date.now(),
51      });
52    },
53  });
54
55  export const remove = mutation({
56    args: { id: v.id("analyses") },
57    handler: async (ctx, args) => {
58      const identity = await ctx.auth.getUserIdentity();
59      if (!identity) throw new Error("Not authenticated");
60      const analysis = await ctx.db.get(args.id);
61      if (!analysis || analysis.userId !== identity.subject) {
62        throw new Error("Not authorized");
63      }
64      await ctx.db.delete(args.id);
65    },
66  });
67
```

```

1  import { motion } from "framer-motion";
2
3  interface CredibilityGaugeProps {
4    confidence: number;
5    verdict: "likely_real" | "likely_fake" | "uncertain";
6    size?: number;
7  }
8
9  const verdictColors: Record<string, { stroke: string; glow: string; text: string }> = {
10    likely_real: {
11      stroke: "#10b981",
12      glow: "rgba(16, 185, 129, 0.3)",
13      text: "text-emerald-600",
14    },
15    uncertain: {
16      stroke: "#f59e0b",
17      glow: "rgba(245, 158, 11, 0.3)",
18      text: "text-amber-600",
19    },
20    likely_fake: {
21      stroke: "#ef4444",
22      glow: "rgba(239, 68, 68, 0.3)",
23      text: "text-red-600",
24    },
25  };
26
27  export function CredibilityGauge({
28    confidence,
29    verdict,
30    size = 180,
31  }: CredibilityGaugeProps) {
32    const colors = verdictColors[verdict];
33    const strokeWidth = 10;
34    const radius = (size - strokeWidth) / 2;
35    const circumference = 2 * Math.PI * radius;
36    const progress = (confidence / 100) * circumference;
37    const center = size / 2;
38
39    return (
40      <div className="relative inline-flex items-center justify-center">
41        <div
42          className="absolute rounded-full blur-xl"
43          style={{
44            width: size * 0.8,
45            height: size * 0.8,
46            background: colors.glow,
47          }}
48        />
49
50
51        <svg
52          width={size}
53          height={size}
54          viewBox={`0 0 ${size} ${size}`}
55          className="relative -rotate-90"
56        >
57          <div>
58            <circle
59              cx={center}
60              cy={center}
61              r={radius}
62              fill="none"
63              stroke="currentColor"
64              strokeWidth={strokeWidth}
65              className="text-muted/30"
66            />
67
68            <div>
69              <motion.circle

```



```

70         cx={center}
71         cy={center}
72         r={radius}
73         fill="none"
74         stroke={colors.stroke}
75         strokeWidth={strokeWidth}
76         strokeLinecap="round"
77         strokeDasharray={circumference}
78         initial={{ strokeDashoffset: circumference }}
79         animate={{ strokeDashoffset: circumference - progress }}
80         transition={{ duration: 1.2, delay: 0.3, ease: [0.22, 1, 0.36, 1] }}
81         style={{ filter: `drop-shadow(0 0 6px ${colors.glow})` }}
82     />
83 </svg>
84
85     {/* Center text */}
86     <div className="absolute inset-0 flex flex-col items-center justify-center">
87         <motion.span
88             initial={{ opacity: 0, scale: 0.5 }}
89             animate={{ opacity: 1, scale: 1 }}
90             transition={{ duration: 0.5, delay: 0.6 }}
91             className="text-3xl font-extrabold tracking-tight"
92         >
93             {confidence}%
94         </motion.span>
95         <motion.span
96             initial={{ opacity: 0 }}
97             animate={{ opacity: 1 }}
98             transition={{ duration: 0.3, delay: 0.8 }}
99             className="text-[11px] text-muted-foreground font-medium"
100         >
101             confidence
102         </motion.span>
103     </div>
104 </div>
105 );
106 }
107

```

```
1  import { motion } from "framer-motion";
2  import {
3    PieChart,
4    Pie,
5    Cell,
6    BarChart,
7    Bar,
8    XAxis,
9    YAxis,
10   Tooltip,
11   ResponsiveContainer,
12 } from "recharts";
13 import {
14   TrendingUp,
15   Shield,
16   AlertTriangle,
17   CheckCircle2,
18   XCircle,
19   BarChart3,
20 } from "lucide-react";
21
22 interface Analysis {
23   verdict: "likely_real" | "likely_fake" | "uncertain";
24   confidence: number;
25   createdAt: number;
26   redFlags: string[];
27   greenFlags: string[];
28 }
29
30 interface StatsViewProps {
31   analyses: Analysis[];
32 }
33
34 const COLORS = {
35   likely_real: "#10b981",
36   uncertain: "#f59e0b",
37   likely_fake: "#ef4444",
38 };
39
40 const VERDICT_LABELS = {
41   likely_real: "Likely Real",
42   uncertain: "Uncertain",
43   likely_fake: "Likely Fake",
44 };
45
46 export function StatsView({ analyses }: StatsViewProps) {
47   if (analyses.length === 0) return null;
48
49   // Calculate stats
50   const totalAnalyses = analyses.length;
51   const verdictCounts = { likely_real: 0, uncertain: 0, likely_fake: 0 };
52   let totalConfidence = 0;
53   let totalRedFlags = 0;
54   let totalGreenFlags = 0;
55
56   for (const a of analyses) {
57     verdictCounts[a.verdict]++;
58     totalConfidence += a.confidence;
59     totalRedFlags += a.redFlags.length;
60     totalGreenFlags += a.greenFlags.length;
61   }
62
63   const avgConfidence = Math.round(totalConfidence / totalAnalyses);
64
65   const pieData = (Object.keys(verdictCounts) as Array<keyof typeof verdictCounts>).map(
66     (key) => ({
67       name: VERDICT_LABELS[key],
68       value: verdictCounts[key],
69       color: COLORS[key],
```

```

70     }},
71     ).filter((d) => d.value > 0);
72
73     // Confidence distribution
74     const confBuckets = [
75       { range: "35-50%", count: 0, fill: "#94a3b8" },
76       { range: "50-65%", count: 0, fill: "#f59e0b" },
77       { range: "65-80%", count: 0, fill: "#3b82f6" },
78       { range: "80-95%", count: 0, fill: "#10b981" },
79     ];
80     for (const a of analyses) {
81       if (a.confidence < 50) confBuckets[0].count++;
82       else if (a.confidence < 65) confBuckets[1].count++;
83       else if (a.confidence < 80) confBuckets[2].count++;
84       else confBuckets[3].count++;
85     }
86
87     const statCards = [
88       {
89         label: "Total Analyses",
90         value: totalAnalyses,
91         icon: BarChart3,
92         color: "text-primary",
93         bg: "bg-primary/10",
94       },
95       {
96         label: "Avg. Confidence",
97         value: `${avgConfidence}%`,
98         icon: TrendingUp,
99         color: "text-blue-600",
100        bg: "bg-blue-500/10",
101      },
102      {
103        label: "Red Flags Found",
104        value: totalRedFlags,
105        icon: AlertTriangle,
106        color: "text-red-600",
107        bg: "bg-red-500/10",
108      },
109      {
110        label: "Green Flags Found",
111        value: totalGreenFlags,
112        icon: CheckCircle2,
113        color: "text-emerald-600",
114        bg: "bg-emerald-500/10",
115      },
116    ];
117
118     return (
119       <div className="space-y-6">
120         {/* Stat cards */}
121         <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
122           {statCards.map((card, i) => (
123             <motion.div
124               key={card.label}
125               initial={{ opacity: 0, y: 10 }}
126               animate={{ opacity: 1, y: 0 }}
127               transition={{ duration: 0.3, delay: i * 0.08 }}
128               className="glass-card rounded-xl p-4"
129             >
130               <div className="flex items-center gap-2 mb-2">
131                 <div className={`w-7 h-7 rounded-lg ${card.bg} flex items-center justify-center`}
132                   <card.icon className={`w-3.5 h-3.5 ${card.color}`} />
133                 </div>
134                 <span className="text-[11px] text-muted-foreground">
135                   {card.label}
136                 </span>
137               </div>
138               <span className="text-xl font-bold">{card.value}</span>
139             </motion.div>
140           )))
141         </div>
142
143         {/* Charts */}

```

```

144     <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
145       {/* Verdict distribution */}
146       {pieData.length > 0 && (
147         <motion.div
148           initial={{ opacity: 0, y: 10 }}
149           animate={{ opacity: 1, y: 0 }}
150           transition={{ duration: 0.3, delay: 0.3 }}
151           className="glass-card rounded-2xl p-6"
152         >
153           <h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wides">
154             Verdict Distribution
155           </h3>
156           <div className="flex items-center justify-center">
157             <ResponsiveContainer width={200} height={200}>
158               <PieChart>
159                 <Pie
160                   data={pieData}
161                   cx="50%"
162                   cy="50%"
163                   innerRadius={55}
164                   outerRadius={80}
165                   dataKey="value"
166                   stroke="none"
167                 >
168                   {pieData.map((entry) => (
169                     <Cell key={entry.name} fill={entry.color} />
170                   ))}
171                 </Pie>
172                 <Tooltip
173                   contentStyle={{
174                     background: "rgba(255,255,255,0.9)",
175                     border: "1px solid rgba(0,0,0,0.1)",
176                     borderRadius: "8px",
177                     fontSize: "12px",
178                   }}
179                 />
180               </PieChart>
181             </ResponsiveContainer>
182           </div>
183           <div className="flex flex-wrap justify-center gap-3 mt-2">
184             {pieData.map((d) => (
185               <div key={d.name} className="flex items-center gap-1.5">
186                 <div
187                   className="w-2.5 h-2.5 rounded-full"
188                   style={{ background: d.color }}
189                 />
190                 <span className="text-[11px] text-muted-foreground">
191                   {d.name} ({d.value})
192                 </span>
193               </div>
194             ))}
195           </div>
196         </motion.div>
197       )}
198
199       {/* Confidence distribution */}
200       {confBuckets.some((b) => b.count > 0) && (
201         <motion.div
202           initial={{ opacity: 0, y: 10 }}
203           animate={{ opacity: 1, y: 0 }}
204           transition={{ duration: 0.3, delay: 0.4 }}
205           className="glass-card rounded-2xl p-6"
206         >
207           <h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wides">
208             Confidence Distribution
209           </h3>
210           <ResponsiveContainer width="100%" height={200}>
211             <BarChart data={confBuckets}>
212               <XAxis
213                 dataKey="range"
214                 tick={{ fontSize: 11, fill: "#94a3b8" }}
215                 axisLine={false}
216                 tickLine={false}
217               />

```

```

218         <YAxis
219             tick={{ fontSize: 11, fill: "#94a3b8" }}
220             axisLine={false}
221             tickLine={false}
222             allowDecimals={false}
223         />
224         <Tooltip
225             contentStyle={{
226                 background: "rgba(255,255,255,0.9)",
227                 border: "1px solid rgba(0,0,0,0.1)",
228                 borderRadius: "8px",
229                 fontSize: "12px",
230             }}
231         />
232         <Bar dataKey="count" radius={[6, 6, 0, 0]}>
233             {confBuckets.map((entry) => (
234                 <Cell key={entry.range} fill={entry.fill} />
235             ))}
236         </Bar>
237     </BarChart>
238 </ResponsiveContainer>
239 </motion.div>
240     )}
241 </div>
242 </div>
243 );
244 }
245

```

```

1  import { motion } from "framer-motion";
2  import {
3    Brain,
4    Layers,
5    Search,
6    BarChart3,
7    Shield,
8    BookOpen,
9    Target,
10   GitBranch,
11 } from "lucide-react";
12
13 const sections = [
14   {
15     icon: Brain,
16     title: "System Architecture",
17     content:
18       "Veritas employs a multi-layered NLP analysis pipeline built on Convex (serverless backen
19     details: [
20       "Serverless Convex backend with real-time data synchronization",
21       "React + TypeScript frontend with Vite for fast builds",
22       "Shadcn/UI component library with custom Glassmorphism theme",
23       "Framer Motion for smooth, meaningful animations",
24     ],
25   },
26   {
27     icon: Layers,
28     title: "Three-Layer Analysis Pipeline",
29     content:
30       "The detection engine operates in three sequential layers: pattern extraction, severity s
31     details: [
32       "Layer 1: Regex-based linguistic pattern extraction with weighted matching",
33       "Layer 2: Category-level severity scoring (red flags vs. green flags)",
34       "Layer 3: Ratio-based verdict calculation with confidence calibration",
35       "Each layer is independently testable and auditable",
36     ],
37   },
38   {
39     icon: Search,
40     title: "Red Flag Detection (12 Categories)",
41     content:
42       "The system identifies 12 distinct categories of misinformation indicators, each with sev
43     details: [
44       "Sensationalist language - emotionally manipulative words and phrases",
45       "Clickbait patterns - common headline manipulation tactics",
46       "Anonymous sourcing - vague attribution without named individuals",
47       "Fear-mongering - alarmist framing designed to provoke anxiety",
48       "Conspiracy language - conspiratorial rhetoric and framing",
49       "Excessive capitalization - non-professional formatting",
50       "Emoji overuse - unprofessional content indicators",
51       "ALL CAPS emphasis - shouting via typography",
52       "Multi-exclamation marks - emotional punctuation abuse",
53       "Urgency language - pressure tactics to force sharing",
54       "Missing citations - no sources, URLs, or references",
55       "Inappropriate length - too short for genuine articles",
56     ],
57   },
58   {
59     icon: Target,
60     title: "Green Flag Detection (9 Categories)",
61     content:
62       "Simultaneously, the system evaluates credibility indicators consistent with established
63     details: [
64       "Named sources with credentials and specific attributions",
65       "Quantitative data - statistics, percentages, dollar amounts",
66       "Credible institutions - Reuters, BBC, universities, journals",
67       "Temporal specificity - exact dates, timelines, event markers",
68       "Balanced reporting - multiple perspectives, counterarguments",
69       "Journalistic structure - who, what, where, when, why framework",

```

```

70     "Neutral tone - absence of emotional or manipulative language",
71     "Appropriate length - 80-800 words, consistent with news articles",
72     "Named quotes - direct attributions to specific individuals",
73   ],
74 },
75 {
76   icon: BarChart3,
77   title: "Scoring Methodology",
78   content:
79     "Verdicts are determined by calculating the ratio of red flag severity scores to green fl
80   details: [
81     "Red flag ratio ? 0.65 ? Likely Fake (confidence 55-95%)",
82     "Green flag ratio ? 0.65 ? Likely Real (confidence 55-95%)",
83     "Ratio difference > 0.1 ? Leaning verdict (confidence 42-78%)",
84     "Balanced scores ? Uncertain (confidence 35-50%)",
85     "Confidence is calibrated to prevent overconfident claims",
86   ],
87 },
88 {
89   icon: GitBranch,
90   title: "Why Rule-Based Over ML?",
91   content:
92     "The system deliberately uses rule-based NLP rather than trained ML models. This design d
93   details: [
94     "Explainability - every decision can be traced to specific patterns",
95     "Transparency - users see exactly why content was flagged",
96     "No training data required - works immediately without labeled datasets",
97     "Interpretability - critical for educational and academic applications",
98     "Consistency - deterministic output for the same input",
99     "Trade-off acknowledged: ML models (BERT, RoBERTa) could achieve higher accuracy but sacr
100   ],
101 },
102 {
103   icon: BookOpen,
104   title: "Academic References",
105   content:
106     "The detection heuristics are informed by established misinformation research:",
107   details: [
108     "MIT Media Lab - Fake News detection using linguistic feature extraction",
109     "First Draft News - Misinformation taxonomy and indicator framework",
110     "Stanford Internet Observatory - Virality Project pattern analysis",
111     "Reuters Institute - Digital News Report credibility framework",
112     "LIAR Dataset (Wang, 2017) - 12.8K labeled statements for NLP research",
113     "MediaWise - Teen media literacy indicators research",
114   ],
115 },
116 {
117   icon: Shield,
118   title: "Limitations & Future Work",
119   content:
120     "The current approach has acknowledged limitations that could be addressed in future vers
121   details: [
122     "Does not cross-reference claims against external fact-check databases",
123     "No image/video analysis (only text-based content)",
124     "Rule-based system requires manual pattern updates",
125     "Future: Fine-tuned BERT classifier on LIAR dataset (12.8K statements)",
126     "Future: TF-IDF vectorization for topic modeling",
127     "Future: Google Fact Check Tools API integration",
128     "Future: Image forensics for manipulated media detection",
129   ],
130 },
131 ];
132
133 export function MethodologyView() {
134   return (
135     <div className="space-y-6">
136       { /* Header */ }
137       <motion.div
138         initial={{ opacity: 0, y: 10 }}
139         animate={{ opacity: 1, y: 0 }}
140         className="glass-card rounded-2xl p-6"
141       >
142         <div className="flex items-center gap-3 mb-3">
143           <div className="w-10 h-10 rounded-xl bg-primary/10 flex items-center justify-center">

```

```

144         <BookOpen className="w-5 h-5 text-primary" />
145     </div>
146     <div>
147         <h2 className="text-lg font-bold">Technical Methodology</h2>
148         <p className="text-sm text-muted-foreground">
149             How Veritas detects misinformation - for academic reference
150         </p>
151     </div>
152 </div>
153 </motion.div>
154
155 /* Sections */
156 {sections.map((section, i) => (
157     <motion.div
158         key={section.title}
159         initial={{ opacity: 0, y: 10 }}
160         animate={{ opacity: 1, y: 0 }}
161         transition={{ duration: 0.3, delay: i * 0.06 }}
162         className="glass-card rounded-2xl p-6"
163     >
164         <div className="flex items-center gap-3 mb-3">
165             <div className="w-9 h-9 rounded-xl bg-primary/10 flex items-center justify-center s
166                 <section.icon className="w-4 h-4 text-primary" />
167             </div>
168             <h3 className="text-base font-bold">{section.title}</h3>
169         </div>
170         <p className="text-sm text-muted-foreground leading-relaxed mb-3">
171             {section.content}
172         </p>
173         <ul className="space-y-1.5">
174             {section.details.map((detail) => (
175                 <li
176                     key={detail}
177                     className="flex items-start gap-2 text-xs text-muted-foreground leading-relaxed
178                 >
179                     <div className="w-1 h-1 rounded-full bg-primary mt-1.5 shrink-0" />
180                     {detail}
181                 </li>
182             ))}
183         </ul>
184     </motion.div>
185 ))}
186 </div>
187 );
188 }
189

```



```
1  import { createContext, useContext, useEffect, useState, type ReactNode } from "react";
2
3  interface ThemeContextType {
4    theme: string;
5    setTheme: (theme: string) => void;
6  }
7
8  const ThemeContext = createContext<ThemeContextType>({ theme: "light", setTheme: () => {} });
9
10 export function useTheme() {
11   return useContext(ThemeContext);
12 }
13
14 export function ThemeProvider({ children }: { children: ReactNode }) {
15   const [theme, setThemeState] = useState("light");
16
17   useEffect(() => {
18     const saved = localStorage.getItem("veritas-theme") || "light";
19     setThemeState(saved);
20     document.documentElement.classList.toggle("dark", saved === "dark");
21   }, []);
22
23   const setTheme = (newTheme: string) => {
24     setThemeState(newTheme);
25     localStorage.setItem("veritas-theme", newTheme);
26     document.documentElement.classList.toggle("dark", newTheme === "dark");
27   };
28
29   return (
30     <ThemeContext.Provider value={{ theme, setTheme }}>
31       {children}
32     </ThemeContext.Provider>
33   );
34 }
35
```

```

1  import { useState, useCallback, useEffect, useRef } from "react";
2  import { motion, AnimatePresence } from "framer-motion";
3  import { useQuery, useMutation, useAction } from "convex/react";
4  import { api } from "@convex/_generated/api";
5  import { useNavigate } from "react-router";
6  import { useTheme } from "@components/ThemeProvider";
7  import { toast } from "sonner";
8  import {
9    Shield, Search, Clock, Home, Loader2, CheckCircle2, AlertTriangle,
10   XCircle, FileText, Link, Trash2, ChevronRight, Brain, BarChart3,
11   Sparkles, ArrowLeft, ClipboardPaste, BookOpen, TrendingUp,
12   Sun, Moon, Download, Share2, Lightbulb, Target,
13 } from "lucide-react";
14 import { Button } from "@components/ui/button";
15 import { Textarea } from "@components/ui/textarea";
16 import { Badge } from "@components/ui/badge";
17 import { CredibilityGauge } from "@components/CredibilityGauge";
18 import { StatsView } from "@components/StatsView";
19 import { MethodologyView } from "@components/MethodologyView";
20
21 type Verdict = "likely_real" | "likely_fake" | "uncertain";
22
23 interface CategoryBreakdown {
24   category: string;
25   type: "red" | "green";
26   score: number;
27   maxScore: number;
28   findings: string[];
29 }
30
31 interface AnalysisResult {
32   verdict: Verdict;
33   confidence: number;
34   summary: string;
35   redFlags: string[];
36   greenFlags: string[];
37   reasoning: string;
38   triggeredKeywords: string[];
39   categoryBreakdown: CategoryBreakdown[];
40   wordCount: number;
41 }
42
43 const verdictConfig: Record<Verdict, { label: string; icon: typeof CheckCircle2; color: string;
44   likely_real: { label: "Likely Real", icon: CheckCircle2, color: "text-emerald-600", bg: "bg-e
45   uncertain: { label: "Uncertain", icon: AlertTriangle, color: "text-amber-600", bg: "bg-amber-
46   likely_fake: { label: "Likely Fake", icon: XCircle, color: "text-red-600", bg: "bg-red-500/10
47 };
48
49 const sampleTexts = [
50   { label: "Scientists Discover New Species", text: "In a groundbreaking discovery, a team of m
51   { label: "Miracle Cure Hidden by Big Pharma", text: "EXPOSED!!! A secret natural cure for ALL
52   { label: "Market Rate Report", text: "The Federal Reserve held interest rates steady at 5.25-
53   { label: "Political Conspiracy Post", text: "WAKE UP SHEEPLE!!! The deep state doesn't want y
54   { label: "Climate Change Report", text: "A comprehensive study published in the journal Scien
55   { label: "Celebrity Health Rumor", text: "SHOCKING!!! Famous Hollywood star secretly DEAD but
56 ];
57
58 const mediaLiteracyTips = [
59   "Always check the source. Is it a recognized news organization with editorial standards?",
60   "Look for named sources. Credible articles cite specific people with their titles and affilia
61   "Be wary of ALL CAPS and excessive exclamation marks - professional journalism avoids these."
62   "Check if the article presents multiple perspectives or just one side of the story.",
63   "Verify statistics by searching for the original study or data source.",
64   "If an article makes you very angry or scared, pause before sharing - that's by design.",
65   "Look for specific dates, locations, and quotes - vague articles are less reliable.",
66   "Share buttons with 'before they delete it' are designed to pressure you into sharing false c
67 ];
68
69 type ViewType = "analyze" | "result" | "history" | "stats" | "methodology";

```

```

70
71 export default function Dashboard() {
72   const navigate = useNavigate();
73   const { theme, setTheme } = useTheme();
74   const analyses = useQuery(api.analyses.listByUser);
75   const createAnalysis = useMutation(api.analyses.create);
76   const deleteAnalysis = useMutation(api.analyses.remove);
77   const runAnalysis = useAction(api.analyzeNews.analyzeNews);
78
79   const [inputText, setInputText] = useState("");
80   const [inputType, setInputType] = useState<"text" | "url">("text");
81   const [isAnalyzing, setIsAnalyzing] = useState(false);
82   const [currentResult, setCurrentResult] = useState<AnalysisResult | null>(null);
83   const [activeView, setActiveView] = useState<ViewType>("analyze");
84   const [currentTip, setCurrentTip] = useState(0);
85   const textareaRef = useRef<HTMLTextAreaElement>(null);
86
87   // Rotate tips
88   useEffect(() => {
89     const interval = setInterval(() => setCurrentTip(t => (t + 1) % mediaLiteracyTips.length),
90     return () => clearInterval(interval);
91   }, []);
92
93   // Keyboard shortcut: Ctrl+Enter to analyze
94   useEffect(() => {
95     const handler = (e: KeyboardEvent) => {
96       if ((e.ctrlKey || e.metaKey) && e.key === "Enter" && activeView === "analyze" && !isAnaly
97         e.preventDefault();
98         handleAnalyze();
99     }
100   });
101   window.addEventListener("keydown", handler);
102   return () => window.removeEventListener("keydown", handler);
103 }, [activeView, isAnalyzing, inputText]);
104
105 const handleAnalyze = useCallback(async () => {
106   if (!inputText.trim()) { toast.error("Please enter some text to analyze."); return; }
107   if (inputText.trim().length < 20) { toast.error("Please enter at least 20 characters."); re
108   setIsAnalyzing(true);
109   setCurrentResult(null);
110   try {
111     const result = await runAnalysis({ text: inputText.trim(), inputType });
112     setCurrentResult(result);
113     setActiveView("result");
114     try {
115       await createAnalysis({
116         inputText: inputText.trim().slice(0, 5000), inputType,
117         verdict: result.verdict, confidence: result.confidence, summary: result.summary,
118         redFlags: result.redFlags, greenFlags: result.greenFlags, reasoning: result.reasoning
119       });
120     } catch { /* best-effort */ }
121     toast.success("Analysis complete!");
122   } catch (error) {
123     toast.error(error instanceof Error ? error.message : "Analysis failed.");
124   } finally { setIsAnalyzing(false); }
125 }, [inputText, inputType, runAnalysis, createAnalysis]);
126
127 const handleDelete = useCallback(async (id: string) => {
128   try { await deleteAnalysis({ id: id as never }); toast.success("Removed from history."); }
129   catch { toast.error("Failed to delete."); }
130 }, [deleteAnalysis]);
131
132 const handleLoadFromHistory = useCallback((analysis: any) => {
133   if (!analysis) return;
134   setCurrentResult({
135     verdict: analysis.verdict, confidence: analysis.confidence, summary: analysis.summary,
136     redFlags: analysis.redFlags, greenFlags: analysis.greenFlags, reasoning: analysis.reasoni
137     triggeredKeywords: analysis.triggeredKeywords ?? [], categoryBreakdown: analysis.category
138     wordCount: analysis.wordCount ?? analysis.inputText.split(/\s+/).length,
139   });
140   setInputText(analysis.inputText); setInputType(analysis.inputType); setActiveView("result")
141 }, []);
142
143 // Split text for keyword highlighting

```

```

144     const getHighlightedParts = useCallback((text: string, keywords: string[]) => {
145         if (!keywords.length || !text) return [{ text, highlighted: false }];
146         const parts: Array<{ text: string; highlighted: boolean }> = [];
147         let remaining = text;
148         for (const kw of keywords) {
149             const idx = remaining.toLowerCase().indexOf(kw.toLowerCase());
150             if (idx >= 0) {
151                 if (idx > 0) parts.push({ text: remaining.slice(0, idx), highlighted: false });
152                 parts.push({ text: remaining.slice(idx, idx + kw.length), highlighted: true });
153                 remaining = remaining.slice(idx + kw.length);
154             }
155         }
156         if (remaining) parts.push({ text: remaining, highlighted: false });
157         return parts.length ? parts : [{ text, highlighted: false }];
158     }, []);
159
160     // Export as text
161     const handleExport = useCallback(() => {
162         if (!currentResult) return;
163         const vc = verdictConfig[currentResult.verdict];
164         const text = `VERITAS ANALYSIS REPORT\n${"=".repeat(40)}\n\nVerdict: ${vc.label}\nConfidenc
165         const blob = new Blob([text], { type: "text/plain" });
166         const url = URL.createObjectURL(blob);
167         const a = document.createElement("a"); a.href = url; a.download = "veritas-analysis.txt"; a
168         URL.revokeObjectURL(url);
169         toast.success("Report downloaded!");
170     }, [currentResult, inputText]);
171
172     // Share via Web Share API
173     const handleShare = useCallback(async () => {
174         if (!currentResult) return;
175         const vc = verdictConfig[currentResult.verdict];
176         const shareData = { title: "Veritas Analysis", text: `${vc.label} (${currentResult.confiden
177         try {
178             if (navigator.share) await navigator.share(shareData);
179             else { await navigator.clipboard.writeText(`${shareData.text}\n${shareData.url}`); toast.
180         } catch { /* user cancelled */ }
181     }, [currentResult]);
182
183     const vc = currentResult ? verdictConfig[currentResult.verdict] : null;
184
185     const navItems = [
186         { key: "analyze" as ViewType, icon: Search, label: "Analyze" },
187         { key: "result" as ViewType, icon: BarChart3, label: "Results", disabled: !currentResult },
188         { key: "history" as ViewType, icon: Clock, label: "History" },
189         { key: "stats" as ViewType, icon: TrendingUp, label: "Stats" },
190         { key: "methodology" as ViewType, icon: BookOpen, label: "Method" },
191     ];
192
193     return (
194         <div className="min-h-screen gradient-bg text-foreground">
195             <div className="fixed inset-0 pointer-events-none overflow-hidden -z-10">
196                 <div className="absolute -top-40 -left-40 w-[500px] h-[500px] rounded-full bg-primary/6
197                 <div className="absolute top-1/3 -right-32 w-[400px] h-[400px] rounded-full bg-chart-2/
198             </div>
199
200             {/* Nav */}
201             <nav className="sticky top-0 z-50 glass-strong border-b border-border/50">
202                 <div className="mx-auto max-w-6xl px-4 sm:px-6 h-14 sm:h-16 flex items-center justify-b
203                     <button type="button" className="cursor-pointer flex items-center gap-2 sm:gap-3" onC
204                         <div className="w-7 h-7 sm:w-8 sm:h-8 rounded-xl bg-primary flex items-center justi
205                         <span className="font-bold tracking-tight hidden sm:inline text-sm">Veritas</span>
206                     </button>
207                     <div className="flex items-center gap-0.5 sm:gap-1">
208                         {navItems.map(item => (
209                             <Button key={item.key} variant={activeView === item.key ? "default" : "ghost"} si
210                                 className={`cursor-pointer gap-1 sm:gap-1.5 text-[11px] sm:text-xs px-2 sm:px-3
211                                 disabled={item.disabled} onClick={() => setActiveView(item.key)}>
212                                 <item.icon className="w-3 h-3 sm:w-3.5 sm:h-3.5" /><span className="hidden md:i
213                             </Button>
214                         ))}
215                     <div className="w-px h-5 bg-border/50 mx-1" />
216                     <Button variant="ghost" size="icon" className="cursor-pointer h-8 w-8" onClick={()
217                         {theme === "dark" ? <Sun className="w-4 h-4" /> : <Moon className="w-4 h-4" />}

```

```

218         </Button>
219         <Button variant="ghost" size="icon" className="cursor-pointer h-8 w-8" onClick={()
220         </div>
221     </div>
222 </nav>
223
224 <main className="mx-auto max-w-4xl px-4 sm:px-6 py-6 sm:py-8">
225     <AnimatePresence mode="wait">
226         {/* --- ANALYZE --- */}
227         {activeView === "analyze" && (
228             <motion.div key="analyze" initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y:
229                 <div className="mb-6 sm:mb-8">
230                     <div className="flex items-center gap-3 mb-2">
231                         <div className="w-10 h-10 rounded-xl bg-primary/10 flex items-center justify-
232                         <div><h1 className="text-xl sm:text-2xl font-bold tracking-tight">Analyze Con
233                         <p className="text-xs sm:text-sm text-muted-foreground">Paste any news arti
234                     </div>
235                 </div>
236
237                 {/* Media literacy tip */}
238                 <motion.div key={currentTip} initial={{ opacity: 0, y: -5 }} animate={{ opacity:
239                     className="glass-card rounded-xl p-3 sm:p-4 mb-4 flex items-start gap-3">
240                     <Lightbulb className="w-4 h-4 text-amber-500 mt-0.5 shrink-0" />
241                     <p className="text-xs sm:text-sm text-muted-foreground leading-relaxed">{medial
242                 </motion.div>
243
244                 <div className="flex gap-2 mb-4">
245                     <Button variant={inputType === "text" ? "default" : "outline"} className={`curs
246                     <Button variant={inputType === "url" ? "default" : "outline"} className={`curso
247                 </div>
248
249                 <div className="glass-card rounded-2xl p-1 mb-4">
250                     <Textarea ref={textareaRef} value={inputText} onChange={e => setInputText(e.tar
251                     placeholder={inputType === "text" ? "Paste a news article, social media post,
252                     className="min-h-[180px] sm:min-h-[200px] border-0 bg-transparent resize-none
253                 </div>
254
255                 <div className="flex items-center justify-between mb-6 sm:mb-8">
256                     <span className="text-xs text-muted-foreground">{inputText.length > 0 ? `${input
257                     <div className="flex gap-2 items-center">
258                         <span className="text-[10px] text-muted-foreground/50 hidden sm:inline">Ctrl+
259                         <Button variant="ghost" size="sm" className="cursor-pointer gap-1.5 text-xs"
260                         <Button variant="ghost" size="sm" className="cursor-pointer text-xs" onClick=
261                     </div>
262                 </div>
263
264                 <div className="mb-6 sm:mb-8">
265                     <p className="text-xs font-medium text-muted-foreground uppercase tracking-wide
266                     <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3">
267                         {sampleTexts.map(sample => (
268                             <button key={sample.label} type="button" className="glass-card rounded-xl p
269                             onClick={() => { setInputText(sample.text); setInputType("text"); }}>
270                             <div className="flex items-center justify-between mb-1">
271                                 <span className="text-xs font-semibold text-primary truncate">{sample.l
272                                 <ChevronRight className="w-3.5 h-3.5 text-muted-foreground group-hover:
273                             </div>
274                             <p className="text-[11px] text-muted-foreground line-clamp-2 leading-rela
275                             <div className="flex items-center gap-2 mt-2">
276                                 <Badge variant="outline" className={`text-[10px] ${sample.type === "rea
277                                 {sample.type === "real" ? "Likely Real" : "Likely Fake"}
278                             </Badge>
279                                 <Badge variant="outline" className="text-[10px] text-muted-foreground">
280                             </div>
281                             </button>
282                         )))
283                     </div>
284                 </div>
285
286                 <Button size="lg" className="cursor-pointer w-full bg-primary hover:bg-primary/90
287                     onClick={handleAnalyze} disabled={isAnalyzing || !inputText.trim()}>
288                     {isAnalyzing ? <<Loader2 className="w-5 h-5 animate-spin" />Analyzing...</> :
289                 </Button>
290             </motion.div>
291         )}

```

```

292
293   { /* --- RESULT --- */}
294   {activeView === "result" && currentResult && vc && (
295     <motion.div key="result" initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y:
296       <div className="flex items-center justify-between mb-6">
297         <button type="button" className="flex items-center gap-1.5 text-sm text-muted-f
298           <ArrowLeft className="w-4 h-4" />New Analysis
299         </button>
300         <div className="flex gap-2">
301           <Button variant="outline" size="sm" className="cursor-pointer gap-1.5 text-xs
302             <Button variant="outline" size="sm" className="cursor-pointer gap-1.5 text-xs
303           </div>
304         </div>
305
306     { /* Verdict card */}
307     <motion.div initial={{ opacity: 0, scale: 0.95 }} animate={{ opacity: 1, scale: 1
308       className={`glass-card rounded-2xl p-6 sm:p-8 border ${vc.border} mb-6`}>
309       <div className="flex flex-col sm:flex-row items-center gap-6">
310         <CredibilityGauge confidence={currentResult.confidence} verdict={currentResul
311         <div className="flex-1 text-center sm:text-left">
312           <div className="flex items-center gap-3 mb-2 justify-center sm:justify-star
313             <div className={`w-10 h-10 rounded-xl ${vc.bg} flex items-center justify-
314               <h2 className={`text-xl sm:text-2xl font-bold ${vc.color}`}>{vc.label}</h
315             </div>
316             <p className="text-sm text-muted-foreground leading-relaxed">{vc.descriptio
317             <div className="flex items-center gap-4 mt-3 justify-center sm:justify-star
318               {currentResult.redFlags.length > 0 && <div className="flex items-center g
319               {currentResult.greenFlags.length > 0 && <div className="flex items-center
320               <span className="text-[10px] text-muted-foreground">{currentResult.wordCo
321             </div>
322           </div>
323         </div>
324         <div className="mt-6">
325           <div className="flex items-center justify-between text-xs mb-1.5"><span class
326             <div className="h-2.5 rounded-full bg-muted/50 overflow-hidden">
327               <motion.div initial={{ width: 0 }} animate={{ width: `${currentResult.conf
328                 transition={{ duration: 0.8, delay: 0.3, ease: [0.22, 1, 0.36, 1] as [num
329                 className={`h-full rounded-full ${currentResult.verdict === "likely_real"
330               </div>
331             </div>
332         </div>
333       </motion.div>
334
335     { /* Category Breakdown Chart */}
336     {currentResult.categoryBreakdown.length > 0 && (
337       <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} tran
338       <div className="flex items-center gap-2 mb-4"><Target className="w-4 h-4 text
339       <div className="space-y-2.5">
340         {currentResult.categoryBreakdown.filter(c => c.maxScore > 0).map((cat, i) =
341         const pct = Math.round((cat.score / cat.maxScore) * 100);
342         return (
343           <motion.div key={cat.category} initial={{ opacity: 0, x: -10 }} animate
344           <div className="flex items-center justify-between text-[11px] mb-1">
345             <span className="font-medium">{cat.category}</span>
346             <span className={cat.type === "red" ? "text-red-500" : "text-emeral
347           </div>
348           <div className="h-1.5 rounded-full bg-muted/30 overflow-hidden">
349             <motion.div initial={{ width: 0 }} animate={{ width: `${pct}%` }}
350               transition={{ duration: 0.6, delay: 0.3 + i * 0.05 }}
351               className={`h-full rounded-full ${cat.type === "red" ? "bg-red-40
352             </div>
353           </div>
354         </motion.div>
355       </div>
356     </div>
357   )}
358
359   { /* Triggered Keywords */}
360   {currentResult.triggeredKeywords.length > 0 && (
361     <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} tran
362     <h3 className="text-xs font-semibold text-muted-foreground uppercase tracking
363     <div className="flex flex-wrap gap-1.5">
364       {currentResult.triggeredKeywords.map(kw => (
365         <Badge key={kw} variant="outline" className="text-[10px] border-red-500/3

```

```

366         ))}
367       </div>
368     </motion.div>
369   )}
370
371   <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} transi
372     <h3 className="text-xs font-semibold text-muted-foreground uppercase tracking-w
373     <p className="text-sm leading-relaxed">{currentResult.summary}</p>
374   </motion.div>
375
376   <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} transi
377     <h3 className="text-xs font-semibold text-muted-foreground uppercase tracking-w
378     <p className="text-sm leading-relaxed text-muted-foreground">{currentResult.rea
379   </motion.div>
380
381   <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-4">
382     <motion.div initial={{ opacity: 0, x: -10 }} animate={{ opacity: 1, x: 0 }} tra
383     <div className="flex items-center gap-2 mb-4">
384       <div className="w-7 h-7 rounded-lg bg-red-500/10 flex items-center justify-
385       <h3 className="text-sm font-semibold text-red-600">Red Flags</h3>
386       {currentResult.redFlags.length > 0 && <Badge variant="outline" className="b
387     </div>
388     {currentResult.redFlags.length === 0 ? <p className="text-xs text-muted-foreg
389     <ul className="space-y-2">{currentResult.redFlags.map((flag, i) => (
390       <li key={i} className="flex items-start gap-2 text-xs leading-relaxed"><X
391     )}</ul>
392   )}
393   </motion.div>
394   <motion.div initial={{ opacity: 0, x: 10 }} animate={{ opacity: 1, x: 0 }} tran
395     <div className="flex items-center gap-2 mb-4">
396       <div className="w-7 h-7 rounded-lg bg-emerald-500/10 flex items-center just
397       <h3 className="text-sm font-semibold text-emerald-600">Green Flags</h3>
398       {currentResult.greenFlags.length > 0 && <Badge variant="outline" className=
399     </div>
400     {currentResult.greenFlags.length === 0 ? <p className="text-xs text-muted-for
401     <ul className="space-y-2">{currentResult.greenFlags.map((flag, i) => (
402       <li key={i} className="flex items-start gap-2 text-xs leading-relaxed"><C
403     )}</ul>
404   )}
405   </motion.div>
406 </div>
407
408   {/* Highlighted content */}
409   <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} transi
410     <h3 className="text-xs font-semibold text-muted-foreground uppercase tracking-w
411     <p className="text-xs text-muted-foreground leading-relaxed max-h-40 overflow-a
412     {currentResult.triggeredKeywords.length > 0
413       ? getHighlightedParts(inputText, currentResult.triggeredKeywords).map((part
414         part.highlighted
415         ? <span key={i} className="bg-red-500/15 text-red-600 font-medium px-
416         : <span key={i}>{part.text}</span>
417       )
418       : inputText
419     }
420   </p>
421   </motion.div>
422 </motion.div>
423 )}
424
425   {/* --- HISTORY --- */}
426   {activeView === "history" && (
427     <motion.div key="history" initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y:
428     <div className="mb-6 sm:mb-8"><div className="flex items-center gap-3 mb-2"><div
429     {!analyses ? <div className="glass-card rounded-2xl p-12 text-center"><Loader2 cl
430     : analyses.length === 0 ? <div className="glass-card rounded-2xl p-12 text-cent
431     : <div className="space-y-3">{analyses.map((analysis, i) => { const avc = verdi
432     <motion.div key={analysis.id} initial={{ opacity: 0, y: 10 }} animate={{ opa
433     className="glass-card rounded-xl p-4 hover:shadow-md transition-all duratio
434     <div className="flex items-start gap-3 sm:gap-4">
435       <div className={`w-9 h-9 sm:w-10 sm:h-10 rounded-xl ${avc.bg} flex items-
436       <div className="flex-1 min-w-0">
437         <div className="flex items-center gap-2 mb-1 flex-wrap">
438           <span className={`text-xs sm:text-sm font-semibold ${avc.color}`}>{av
439           <Badge variant="outline" className={`text-[10px] ${avc.border} ${avc.

```

```

440         <Badge variant="outline" className="text-[10px] ml-auto">{analysis.in
441         </div>
442         <p className="text-xs text-muted-foreground line-clamp-1 mb-1">{analysi
443         <p className="text-[11px] text-muted-foreground/60 line-clamp-1">{analy
444         </div>
445         <div className="flex items-center gap-1 shrink-0">
446             <Button variant="ghost" size="icon" className="cursor-pointer h-7 w-7 o
447             <ChevronRight className="w-4 h-4 text-muted-foreground group-hover:text
448         </div>
449         </div>
450         </motion.div>
451         ); }</div>
452     }
453 </motion.div>
454 })
455
456 /* --- STATS --- */
457 {activeView === "stats" && (
458     <motion.div key="stats" initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0
459     <div className="mb-6 sm:mb-8"><div className="flex items-center gap-3 mb-2"><div
460     {!analyses ? <div className="glass-card rounded-2xl p-12 text-center"><Loader2 cl
461     : analyses.length === 0 ? <div className="glass-card rounded-2xl p-12 text-cent
462     : <StatsView analyses={analyses} />}
463     </motion.div>
464 ))
465
466 /* --- METHODOLOGY --- */
467 {activeView === "methodology" && (
468     <motion.div key="methodology" initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1
469     <div className="mb-6 sm:mb-8"><div className="flex items-center gap-3 mb-2"><div
470     <MethodologyView />
471     </motion.div>
472 ))
473 </AnimatePresence>
474 </main>
475 </div>
476 );
477 }
478

```